```
    }
    @Scheduled(fixedDelayString = "1h", initialDelayString = "10m")
    public void cleanupExpired() {
        for (var upload : uploads.expired()) {
            deleteUploadFiles(upload);
            uploads.expired(upload.id());
        }
    }
    private BookUpload saveCover(BookUploadModels.UploadEntity upload, byte[] jpeg, CoverSource source, U
UID actor) {
        LibraryRoot root = writableRoot(upload.libraryRootId(), jpeg.length);
        Path rootPath = rootPath(root);
        String relative = ".bookkin-staging/uploads/" + upload.id() + "/cover.jpg";
        Path target = internalTarget(rootPath, relative);
        try {
            ensureInternalDirectory(rootPath, target.getParent());
            try (FileChannel channel = FileChannel.open(target, StandardOpenOption.CREATE, StandardOpenOp
tion.TRUNCATE_EXISTING, StandardOpenOption.WRITE)) {
                channel.write(ByteBuffer.wrap(jpeg));
                channel.force(true);
            }
            uploads.selectCover(upload.id(), source, relative);
            audit.record(actor, "BOOK_UPLOAD_COVER_SELECTED", "BOOK_UPLOAD", upload.id().toString(), uplo
ad.originalFilename(), relative,
                    upload.fingerprint(), fingerprints.sha256(target), "SUCCEEDED", "{\"source\":\"" + so
urce + "\"}");
            return uploads.findOwned(upload.id(), actor).orElseThrow().view();
        } catch (Exception exception) {
            throw new ApiException(HttpStatus.BAD_REQUEST, "COVER_SAVE_FAILED", safeMessage(exception));
        }
    }
    private byte[] normalizeCover(byte[] bytes) {
        if (bytes == null || bytes.length == 0 || bytes.length > properties.upload().maxCoverSize()) {
            throw new ApiException(HttpStatus.PAYLOAD_TOO_LARGE, "COVER_SIZE_LIMIT", "封面为空或超过大小
限制。");
        }
        try {
            BufferedImage source = ImageIO.read(new ByteArrayInputStream(bytes));
            if (source == null || (long) source.getWidth() * source.getHeight() > 80_000_000L || source.g
etWidth() > 12_000 || source.getHeight() > 12_000) {
                throw new IllegalArgumentException("封面格式不支持或像素过大");
            }
            double targetRatio = 2.0 / 3.0;
            int cropWidth = source.getWidth();
            int cropHeight = source.getHeight();
            if ((double) cropWidth / cropHeight > targetRatio) cropWidth = (int) Math.round(cropHeight *
targetRatio);
            else cropHeight = (int) Math.round(cropWidth / targetRatio);
            int x = (source.getWidth() - cropWidth) / 2;
            int y = (source.getHeight() - cropHeight) / 2;
            BufferedImage target = new BufferedImage(800, 1200, BufferedImage.TYPE_INT_RGB);
            Graphics2D graphics = target.createGraphics();
            graphics.setRenderingHint(RenderingHints.KEY_INTERPOLATION, RenderingHints.VALUE_INTERPOLATIO
N_BICUBIC);
            graphics.drawImage(source, 0, 0, 800, 1200, x, y, x + cropWidth, y + cropHeight, null);
            graphics.dispose();
            ByteArrayOutputStream output = new ByteArrayOutputStream();
            ImageIO.write(target, "JPEG", output);
```

```
            return output.toByteArray();
        } catch (Exception exception) {
            throw new ApiException(HttpStatus.BAD_REQUEST, "INVALID_COVER", safeMessage(exception));
        }
    }
    private void validateDraft(MetadataDraft draft, BookFormat format) {
        if (draft == null || draft.title() == null || draft.title().isBlank() || draft.title().length() >
 500) throw new IllegalArgumentException("书名不能为空且不能超过500字");
        if (draft.authors() == null || clean(draft.authors()).isEmpty() || draft.authors().size() > 20) t
hrow new IllegalArgumentException("至少需要一位作者，最多20位");
        if (draft.translators() != null && draft.translators().size() > 20) throw new IllegalArgumentExce
ption("译者最多20位");
        if (draft.tags() != null && draft.tags().size() > 50) throw new IllegalArgumentException("标签最
多50个");
        validateExtension(draft.targetPath(), format);
    }
    private void validateExtension(String path, BookFormat format) {
        if (path == null || !path.toLowerCase(Locale.ROOT).endsWith("." + format.name().toLowerCase(Local
e.ROOT))) {
            throw new IllegalArgumentException("目标文件扩展名必须与书籍格式一致");
        }
    }
    private void markChanges(Map<String, MetadataSource> sources, MetadataDraft before, MetadataDraft aft
er) {
        mark(sources, "title", before.title(), after.title());
        mark(sources, "subtitle", before.subtitle(), after.subtitle());
        mark(sources, "authors", before.authors(), clean(after.authors()));
        mark(sources, "translators", before.translators(), clean(after.translators()));
        mark(sources, "language", before.language(), after.language());
        mark(sources, "publisher", before.publisher(), after.publisher());
        mark(sources, "publishedDate", before.publishedDate(), after.publishedDate());
        mark(sources, "isbn", before.isbn(), after.isbn());
        mark(sources, "description", before.description(), after.description());
        mark(sources, "series", before.series(), after.series());
        mark(sources, "seriesIndex", before.seriesIndex(), after.seriesIndex());
        mark(sources, "tags", before.tags(), clean(after.tags()));
    }
    private void mark(Map<String, MetadataSource> sources, String field, Object before, Object after) {
        MetadataSource requested = sources.get(field);
        if (!Objects.equals(before, after) && requested != MetadataSource.OPEN_LIBRARY && requested != Me
tadataSource.GOOGLE_BOOKS
                && requested != MetadataSource.AI) {
            sources.put(field, MetadataSource.MANUAL);
        }
    }
    private LibraryRoot writableRoot(UUID id, long requiredBytes) {
        LibraryRoot root = roots.findById(id).orElseThrow(() -> ApiException.notFound("LIBRARY_ROOT_NOT_F
OUND", "未找到书库根目录。"));
        if (root.status() != LibraryRoot.RootStatus.ONLINE || !root.canWrite() || !root.canStage()) {
            throw ApiException.conflict("ROOT_READ_ONLY", "书库当前不可写或暂存区不可用。");
        }
        if (root.freeBytes() != null && root.freeBytes() < requiredBytes + 64L * 1024 * 1024) {
            throw ApiException.conflict("INSUFFICIENT_SPACE", "书库剩余空间不足。");
        }
        return root;
    }
    private Path rootPath(LibraryRoot root) {
        try { return Path.of(root.canonicalPath() == null ? root.configuredPath() : root.canonicalPath())
.toRealPath(); }
```

```
        catch (Exception exception) { throw ApiException.conflict("ROOT_OFFLINE", "书库根目录暂时不可访问
。"); }
    }
    private Path internalTarget(Path root, String relative) {
        if (relative == null || !(relative.startsWith(".bookkin-staging/uploads/") || relative.startsWith
(".bookkin-assets/covers/"))) {
            throw new IllegalArgumentException("内部路径无效");
        }
        Path target = root.resolve(relative).normalize();
        if (!target.startsWith(root)) throw new IllegalArgumentException("内部路径越过书库");
        return target;
    }
    private void ensureInternalDirectory(Path root, Path directory) throws Exception {
        Path relative = root.relativize(directory.normalize());
        Path current = root;
        for (Path segment : relative) {
            current = current.resolve(segment);
            if (Files.exists(current, LinkOption.NOFOLLOW_LINKS)) {
                if (Files.isSymbolicLink(current) || !Files.isDirectory(current, LinkOption.NOFOLLOW_LINK
S)) throw new IllegalArgumentException("目标目录不安全");
            } else Files.createDirectory(current);
            if (!current.toRealPath().startsWith(root)) throw new IllegalArgumentException("目标目录越过
书库");
        }
    }
    private void compensate(Path staged, Path target, boolean bookMoved, Path stagedCover, Path asset, bo
olean coverMoved) {
        try { if (bookMoved && target != null && Files.exists(target) && !Files.exists(staged)) Files.mov
e(target, staged, StandardCopyOption.ATOMIC_MOVE); }
        catch (Exception ignored) {}
        try { if (coverMoved && asset != null && stagedCover != null && Files.exists(asset) && !Files.exi
sts(stagedCover)) Files.move(asset, stagedCover, StandardCopyOption.ATOMIC_MOVE); }
        catch (Exception ignored) {}
    }
    private Path targetOrNull(Path root, String relative) {
        try { return root.resolve(paths.normalizeRelative(relative)).normalize(); }
        catch (Exception ignored) { return null; }
    }
    private void deleteUploadFiles(BookUploadModels.UploadEntity upload) {
        try {
            LibraryRoot root = roots.findById(upload.libraryRootId()).orElseThrow();
            Path rootPath = rootPath(root);
            Path directory = internalTarget(rootPath, ".bookkin-staging/uploads/" + upload.id());
            if (Files.isDirectory(directory, LinkOption.NOFOLLOW_LINKS) && !Files.isSymbolicLink(director
y)) {
                try (var stream = Files.list(directory)) { for (Path file : stream.toList()) deleteQuietl
y(file); }
                Files.deleteIfExists(directory);
            }
        } catch (Exception ignored) {}
    }
    private BookUploadModels.UploadEntity owned(UUID id, UUID actor) {
        return uploads.findOwned(id, actor).orElseThrow(() -> ApiException.notFound("UPLOAD_NOT_FOUND", "
未找到这个上传会话。"));
    }
    private BookUploadModels.UploadEntity requireReady(BookUploadModels.UploadEntity upload) {
        if (upload.status() != BookUploadStatus.READY_FOR_REVIEW) throw ApiException.conflict("UPLOAD_STA
TE", "书籍尚未进入可校对状态。");
        return upload;
    }
    private UUID actor(Principal principal) { return users.findByUsername(principal.getName()).orElseThro
w().id(); }
```

```
    private BookFormat format(String filename) {
        String lower = basename(filename).toLowerCase(Locale.ROOT);
        if (lower.endsWith(".epub")) return BookFormat.EPUB;
        if (lower.endsWith(".pdf")) return BookFormat.PDF;
        throw new ApiException(HttpStatus.UNSUPPORTED_MEDIA_TYPE, "UNSUPPORTED_BOOK_FORMAT", "只支持 EPUB
 和 PDF，不支持压缩包。");
    }
    private String basename(String value) {
        if (value == null || value.isBlank() || value.indexOf('\0') >= 0) throw new IllegalArgumentExcept
ion("文件名无效");
        String name = value.replace('\\', '/');
        name = name.substring(name.lastIndexOf('/') + 1).strip();
        if (name.isBlank() || name.length() > 500) throw new IllegalArgumentException("文件名无效");
        return name;
    }
    private List<String> clean(List<String> values) { return values == null ? List.of() : values.stream()
.filter(value -> value != null && !value.isBlank()).map(String::strip).distinct().toList(); }
    private String blank(String value) { return value == null || value.isBlank() ? null : value.strip();
}
    private String relative(Path root, Path target) { return root.relativize(target).toString().replace('
\\', '/'); }
    private static boolean isContentClaimConflict(Throwable error) {
        for (Throwable cause = error; cause != null; cause = cause.getCause()) {
            if (cause instanceof SQLException sql && "23505".equals(sql.getSQLState())
                    && sql.getMessage() != null && sql.getMessage().contains("DUPLICATE_FINGERPRINT")) re
turn true;
        }
        return false;
    }
    private String safeMessage(Exception exception) { String value = exception.getMessage(); return value
 == null || value.isBlank() ? "操作失败" : value.substring(0, Math.min(500, value.length())); }
    private String safeAudit(String value) {
        if (value == null) return "";
        String clean = value.replaceAll("[^a-zA-Z0-9_.-]", "");
        return clean.substring(0, Math.min(80, clean.length()));
    }
    private void deleteQuietly(Path path) { try { Files.deleteIfExists(path); } catch (Exception ignored)
 {} }
    private void cleanupEmpty(Path directory) { try { Files.deleteIfExists(directory); } catch (Exception
 ignored) {} }
    public record CoverLocation(Path path, String mimeType) {}
}
import { Alert } from "@/ui/feedback";
import { RefreshRounded } from "@/ui/icons";
import { Button } from "@/ui/buttons";
import { Chip } from "@/ui/feedback";
import { CircularProgress } from "@/ui/feedback";
import { Stack } from "@/ui/primitives";
import { Table } from "@/ui/primitives";
import { TableBody } from "@/ui/primitives";
import { TableCell } from "@/ui/primitives";
import { TableContainer } from "@/ui/primitives";
import { TableHead } from "@/ui/primitives";
import { TableRow } from "@/ui/primitives";
import { Typography } from "@/ui/primitives";
import { useQuery } from "@tanstack/react-query";
import { api } from "../api/client";
import { ActionToolbar, PageContainer } from "../components/PageHeader";
import type { FileOperationStatus, FileOperationType } from "../domain/types";
```

```
import { tokens } from "../theme/generated-tokens";
const operationLabels: Record<FileOperationType, string> = {
  RENAME: "重命名",
  MOVE: "移动",
  TRASH: "移入回收站",
  RESTORE: "恢复",
  WRITE_METADATA: "写回元数据",
  PURGE: "永久清理",
};
const statusMeta: Record<FileOperationStatus, { label: string; color: "default" | "success" | "warning" |
 "error" }> = {
  PLANNED: { label: "等待执行", color: "default" },
  RUNNING: { label: "执行中", color: "warning" },
  SUCCEEDED: { label: "已完成", color: "success" },
  FAILED: { label: "失败", color: "error" },
  ROLLED_BACK: { label: "已回滚", color: "warning" },
};
const runningStageLabels: Record<string, string> = {
  ATOMIC_MOVE: "正在移动",
  CHECKING_SPACE: "检查空间",
  COPYING: "正在复制",
  VERIFYING_SHA256: "校验文件",
  FINALIZING_TARGET: "正在落位",
  TRASHING_SOURCE: "清理源文件",
  ATOMIC_TRASH: "移入回收站",
  PREPARING_OVERWRITE: "准备写回",
};
const formatCreatedAt = (createdAt: string) => new Intl.DateTimeFormat("zh-CN", {
  month: "short",
  day: "numeric",
  hour: "2-digit",
  minute: "2-digit",
  hour12: false,
}).format(new Date(createdAt));
export function FileOperationsPage() {
  const query = useQuery({ queryKey: ["file-operations"], queryFn: api.listFileOperations, refetchInterva
l: 5000 });
  return (
    <PageContainer>
      <Stack sx={{ mb: 2 }}><ActionToolbar><Button startIcon={<RefreshRounded />} variant="outlined" onCl
ick={() => query.refetch()}>刷新</Button></ActionToolbar></Stack>
      {query.isError ? <Alert severity="error">无法读取操作日志，请刷新重试。</Alert> : query.isPending ?
 (
        <Stack sx={{ py: `${tokens.spacing[20]}px`, alignItems: "center" }}><CircularProgress /></Stack>
      ) : (query.data ?? []).length === 0 ? (
        <Stack spacing={tokens.spacing[1] / tokens.meta.baseUnit} sx={{ py: `${tokens.spacing[24]}px`, al
ignItems: "center" }}>
          <Typography variant="h4">暂无操作日志</Typography>
          <Typography color="text.secondary">从藏书卡片的更多菜单发起安全预览。</Typography>
        </Stack>
      ) : (
        <TableContainer
          sx={{
            bgcolor: "background.paper",
            border: 1,
```

```
            borderColor: "divider",
            borderRadius: `${tokens.radius.xl}px`,
            overflow: "hidden",
          }}
        >
          <Table aria-label="操作日志列表">
            <TableHead>
              <TableRow sx={{ bgcolor: "background.default" }}>
                <TableCell sx={{ width: "19%" }}>任务</TableCell>
                <TableCell>文件变更</TableCell>
                <TableCell sx={{ width: "15%" }}>状态</TableCell>
                <TableCell align="right" sx={{ width: "17%" }}>时间</TableCell>
              </TableRow>
            </TableHead>
            <TableBody>
              {(query.data ?? []).map((operation) => {
                const status = statusMeta[operation.status];
                const stage = operation.status === "RUNNING" && operation.stage
                  ? runningStageLabels[operation.stage] ?? "正在处理"
                  : undefined;
                const hasDistinctTarget = operation.targetPath && operation.targetPath !== operation.sour
cePath;
                return (
                  <TableRow key={operation.id} hover>
                    <TableCell>
                      <Typography sx={{ fontWeight: tokens.typography.fontWeight.semibold }}>
                        {operationLabels[operation.type]}
                      </Typography>
                    </TableCell>
                    <TableCell sx={{ minWidth: 0 }}>
                      <Typography variant="body2" title={operation.sourcePath} sx={{ overflowWrap: "anywh
ere" }}>
                        {operation.sourcePath}
                      </Typography>
                      {hasDistinctTarget && (
                        <Stack
                          direction="row"
                          spacing={tokens.spacing[2] / tokens.meta.baseUnit}
                          sx={{ mt: `${tokens.spacing[1]}px`, alignItems: "baseline", color: "text.second
ary" }}
                        >
                          <Typography component="span" variant="caption" aria-hidden="true">→</Typograph
y>
                          <Typography
                            component="span"
                            variant="caption"
                            title={operation.targetPath}
                            sx={{ overflowWrap: "anywhere" }}
                          >
                            {operation.targetPath}
                          </Typography>
                        </Stack>
                      )}
                    </TableCell>
```

```
                    <TableCell>
                      <Stack spacing={tokens.spacing[1] / tokens.meta.baseUnit} sx={{ alignItems: "flex-s
tart" }}>
                        <Chip size="small" variant="outlined" color={status.color} label={status.label} /
>
                        {stage && <Typography variant="caption" color="text.secondary">{stage}</Typograph
y>}
                      </Stack>
                    </TableCell>
                    <TableCell align="right">
                      <Typography variant="body2" color="text.secondary" sx={{ whiteSpace: "nowrap" }}>
                        {formatCreatedAt(operation.createdAt)}
                      </Typography>
                    </TableCell>
                  </TableRow>
                );
              })}
            </TableBody>
          </Table>
        </TableContainer>
      )}
    </PageContainer>
  );
}
import { Skeleton } from "@/ui/antd";
import { AutoAwesomeOutlined } from "@/ui/icons";
import { BookmarkAddOutlined } from "@/ui/icons";
import { EditNoteRounded } from "@/ui/icons";
import { FilterListRounded } from "@/ui/icons";
import { KeyboardArrowUpRounded } from "@/ui/icons";
import { MenuBookRounded } from "@/ui/icons";
import { CloudUploadOutlined } from "@/ui/icons";
import { ScheduleRounded } from "@/ui/icons";
import { Alert } from "@/ui/feedback";
import { Box } from "@/ui/primitives";
import { Button } from "@/ui/buttons";
import { Chip } from "@/ui/feedback";
import { Dialog } from "@/ui/overlays";
import { DialogActions } from "@/ui/overlays";
import { DialogContent } from "@/ui/overlays";
import { Divider } from "@/ui/feedback";
import { Fab } from "@/ui/buttons";
import { FormControl } from "@/ui/primitives";
import { MenuItem } from "@/ui/primitives";
import { Select } from "@/ui/forms";
import { Snackbar } from "@/ui/primitives";
import { Stack } from "@/ui/primitives";
import { Tooltip } from "@/ui/feedback";
import { Typography } from "@/ui/primitives";
import { useInfiniteQuery, useQuery, useQueryClient } from "@tanstack/react-query";
import { useCallback, useDeferredValue, useEffect, useMemo, useState, type ReactNode } from "react";
import { useNavigate, useSearchParams } from "react-router-dom";
import { api } from "../api/client";
```

```
import { useAuth } from "../auth/AuthContext";
import { BookCard } from "../components/BookCard";
import { AddToBooklistDialog } from "../components/AddToBooklistDialog";
import { BookUploadDialog } from "../components/BookUploadDialog";
import { FileOperationDialog } from "../components/FileOperationDialog";
import { MetadataDialog } from "../components/MetadataDialog";
import { OverviewCardHeader } from "../components/OverviewCardHeader";
import { OverviewEmptyState } from "../components/OverviewEmptyState";
import { flattenUniquePaginatedItems, PaginatedItemReveal } from "../components/PaginatedItemReveal";
import { PageContainer } from "../components/PageHeader";
import { ReadingStatsPanel } from "../components/ReadingStatsPanel";
import { RecentAnnotationsPanel } from "../components/RecentAnnotationsPanel";
import type { Book, FileOperationType } from "../domain/types";
import { useInfiniteScrollTrigger } from "../hooks/useInfiniteScrollTrigger";
import { tokens } from "../theme/generated-tokens";
type SortKey = "recent" | "title" | "author";
export function LibraryPage() {
  const navigate = useNavigate();
  const queryClient = useQueryClient();
  const { user } = useAuth();
  const [params, setParams] = useSearchParams();
  const [selectedBook, setSelectedBook] = useState<Book | null>(null);
  const [operation, setOperation] = useState<{ book: Book; type: FileOperationType } | null>(null);
  const [metadataBook, setMetadataBook] = useState<Book | null>(null);
  const [booklistBook, setBooklistBook] = useState<Book | null>(null);
  const [uploadOpen, setUploadOpen] = useState(false);
  const [showBackToTop, setShowBackToTop] = useState(false);
  const [notice, setNotice] = useState("");
  const query = params.get("q")?.trim().toLocaleLowerCase("zh-CN") ?? "";
  const deferredQuery = useDeferredValue(query);
  const format = params.get("format") ?? "ALL";
  const sort = (params.get("sort") ?? "recent") as SortKey;
  const canManage = user?.role === "OWNER" || user?.role === "ADMIN";
  const booksQuery = useInfiniteQuery({
    queryKey: ["books", deferredQuery, format, sort],
    initialPageParam: undefined as string | undefined,
    queryFn: ({ pageParam, signal }) => api.listBooks({
      q: deferredQuery || undefined,
      format: format === "ALL" ? undefined : format as "EPUB" | "PDF",
      sort,
      cursor: pageParam,
      limit: 36,
      signal,
    }),
    getNextPageParam: (lastPage) => lastPage.nextCursor,
    refetchInterval: 15_000,
  });
  const readingStatsQuery = useQuery({ queryKey: ["reading-stats", "weekly"], queryFn: api.getWeeklyReadi
ngStats });
  useEffect(() => {
    const updateBackToTopVisibility = () => {
```

```
      const nextVisible = window.scrollY > tokens.spacing[20];
      setShowBackToTop((currentVisible) => currentVisible === nextVisible ? currentVisible : nextVisible)
;
    };
    updateBackToTopVisibility();
    window.addEventListener("scroll", updateBackToTopVisibility, { passive: true });
    return () => window.removeEventListener("scroll", updateBackToTopVisibility);
  }, []);
  const loadedBookEntries = useMemo(() => flattenUniquePaginatedItems(booksQuery.data?.pages), [booksQuer
y.data]);
  const loadedBooks = useMemo(() => loadedBookEntries.map(({ item }) => item), [loadedBookEntries]);
  const loadNextPage = useCallback(() => {
    if (booksQuery.hasNextPage && !booksQuery.isFetchingNextPage) void booksQuery.fetchNextPage();
  }, [booksQuery.fetchNextPage, booksQuery.hasNextPage, booksQuery.isFetchingNextPage]);
  const { triggerRef, supported: automaticLoadingSupported } = useInfiniteScrollTrigger({
    enabled: Boolean(booksQuery.hasNextPage && !booksQuery.isFetchingNextPage && !booksQuery.isFetchNextP
ageError),
    onLoadMore: loadNextPage,
  });
  const bookEntries = useMemo(() => loadedBookEntries.filter(({ item }) => item.status === "AVAILABLE"),
[loadedBookEntries]);
  const books = useMemo(() => bookEntries.map(({ item }) => item), [bookEntries]);
  const updateParam = (key: string, value: string, defaultValue?: string) => {
    const next = new URLSearchParams(params);
    if (!value || value === defaultValue) next.delete(key);
    else next.set(key, value);
    setParams(next, { replace: true });
  };
  const reading = loadedBooks.find((book) => book.progress > 0);
  const featured = books[0] ?? reading;
  return (
    <>
      <PageContainer sx={{ pt: { xs: 3, md: 5 } }}>
        <Box
          component="section"
          aria-label="书库概览"
          sx={{
            display: "grid",
            gridTemplateColumns: {
              xs: "minmax(0, 1fr)",
              sm: "repeat(2, minmax(0, 1fr))",
            },
            // Keep the four overview cards on a shared two-row rhythm.  A
            // fixed token-derived track prevents the compact empty states
            // from collapsing while the featured cover and weekly chart set
            // the height of the upper row.
            gridAutoRows: `${tokens.spacing[24] * 3.5}px`,
            gap: { xs: `${tokens.spacing[4]}px`, sm: `${tokens.spacing[6]}px` },
            mb: { xs: `${tokens.spacing[6]}px`, md: `${tokens.spacing[8]}px` },
            alignItems: "stretch",
          }}
        >
          {booksQuery.isPending ? Array.from({ length: 4 }, (_, index) => (
            <Box key={index} role="status" aria-label="正在加载书库概览" sx={{ p: 3, border: 1, borderCol
or: "divider", borderRadius: `${tokens.radius.xl}px`, bgcolor: "background.paper" }}>
```

```
              <Skeleton active paragraph={{ rows: 4 }} />
            </Box>
          )) : <>
          <>
              <Box
                component="article"
                aria-labelledby="featured-book-title"
                sx={{
                  minWidth: 0,
                  gridColumn: { sm: 1 },
                  gridRow: { sm: 1 },
                  display: "flex",
                  flexDirection: "column",
                  p: { xs: `${tokens.spacing[6]}px`, lg: `${tokens.spacing[8]}px` },
                  bgcolor: "background.paper",
                  border: 1,
                  borderColor: "divider",
                  borderRadius: `${tokens.radius.xl}px`,
                  overflow: "hidden",
                }}
              >
                <OverviewCardHeader id="featured-card-title" icon={<AutoAwesomeOutlined />}>本周新藏</Ove
rviewCardHeader>
                {featured ? <Box
                  sx={{
                    display: "grid",
                    gridTemplateColumns: "minmax(0, 0.92fr) minmax(0, 1.08fr)",
                    gap: { xs: `${tokens.spacing[4]}px`, md: `${tokens.spacing[6]}px` },
                    flex: 1,
                    minHeight: 0,
                    mt: `${tokens.spacing[4]}px`,
                  }}
                >
                  <Box sx={{ display: "flex", alignItems: "center", justifyContent: "center", minWidth: 0
, minHeight: 0, containerType: "size" }}>
                    <Box component="img" src={featured.coverUrl} alt={`${featured.title}封面`} loading="e
ager" decoding="async" fetchPriority="high" sx={{ display: "block", width: "min(100cqw, calc(100cqh * 2 /
 3))", height: "auto", aspectRatio: "2 / 3", objectFit: "cover", borderRadius: `${tokens.radius.lg}px` }}
 />
                  </Box>
                  <Stack sx={{ minWidth: 0, justifyContent: "center", alignItems: "flex-start" }}>
                    <Typography id="featured-book-title" variant="h4" component="h3">{featured.title}</Ty
pography>
                    <Typography color="text.secondary" sx={{ mt: `${tokens.spacing[2]}px` }}>{featured.au
thor}</Typography>
                    <Stack direction="row" sx={{ flexWrap: "wrap", gap: `${tokens.spacing[3]}px`, mt: `${
tokens.spacing[5]}px` }}>
                      <Button variant="contained" startIcon={<MenuBookRounded />} onClick={() => navigate
(`/reader/${featured.id}`)}>开始阅读</Button>
                    </Stack>
                  </Stack>
                </Box> : <OverviewEmptyState titleId="featured-book-title" title="还没有藏书" description
="添加第一本书后，新藏会显示在这里。" />}
              </Box>
              <Box sx={{ minWidth: 0, gridColumn: { sm: 1 }, gridRow: { sm: 2 }, display: "flex", "& > *"
: { flex: 1 } }}>
                <RecentAnnotationsPanel embedded />
              </Box>
          </>
          <>
            <Box sx={{ minWidth: 0, gridColumn: { sm: 2 }, gridRow: { sm: 1 }, display: "flex", "& > *":
{ flex: 1 } }}>
```

```
              <ReadingStatsPanel stats={readingStatsQuery.data} loading={readingStatsQuery.isPending} err
or={readingStatsQuery.isError} />
            </Box>
            <Stack
              component="article"
              aria-labelledby="currently-reading-title"
              sx={{
                gridColumn: { sm: 2 },
                gridRow: { sm: 2 },
                justifyContent: "flex-start",
                p: { xs: `${tokens.spacing[6]}px`, lg: `${tokens.spacing[8]}px` },
                bgcolor: "secondary.dark",
                color: "common.white",
                borderRadius: `${tokens.radius.xl}px`,
              }}
            >
              <OverviewCardHeader id="reading-progress-card-title" icon={<MenuBookRounded />} inverse>阅
读进度</OverviewCardHeader>
              {reading ? (
                <>
                  <Stack sx={{ minWidth: 0, flex: 1, gap: `${tokens.spacing[3]}px`, mt: `${tokens.spacing
[4]}px` }}>
                    <Typography id="currently-reading-title" variant="h4">{reading.title}</Typography>
                    <Typography sx={{ opacity: 0.68 }}>{reading.author}</Typography>
                    <Box sx={{ mt: `${tokens.spacing[2]}px` }}>
                      <Box sx={{ height: `${tokens.spacing[1]}px`, bgcolor: "rgba(255,255,255,.18)", mb:
`${tokens.spacing[2]}px`, borderRadius: `${tokens.radius.pill}px`, overflow: "hidden" }}>
                        <Box sx={{ width: `${reading.progress}%`, height: "100%", bgcolor: "primary.light
" }} />
                      </Box>
                      <Stack direction="row" sx={{ justifyContent: "space-between", opacity: 0.64 }}>
                        <Typography variant="caption">已读 {reading.progress}%</Typography>
                        <Typography variant="caption">第 7 章</Typography>
                      </Stack>
                    </Box>
                  </Stack>
                  <Button
                    variant="contained"
                    onClick={() => navigate(`/reader/${reading.id}`)}
                    sx={{ mt: "auto", bgcolor: "background.paper", color: "text.primary", "&:hover": { bg
color: "background.default" } }}
                  >
                    继续阅读
                  </Button>
                </>
              ) : (
                <OverviewEmptyState
                  titleId="currently-reading-title"
                  title="请开始阅读"
                  description="打开任意一本书后，阅读进度会显示在这里。"
                  inverse
                />
              )}
            </Stack>
          </>
          </>}
```

```
        </Box>
        <Stack sx={{ alignItems: "stretch", gap: 2, mb: 3 }}>
          <Box sx={{ minWidth: 0 }}>
            <Typography variant="h3">全部藏书</Typography>
            {query && <Typography variant="body2" color="text.secondary" sx={{ mt: 0.5 }}>找到 {books.len
gth} 本相关藏书</Typography>}
          </Box>
          <Stack
            direction={{ xs: "column", sm: "row" }}
            spacing={1.5}
            sx={{
              width: "100%",
              justifyContent: "flex-end",
              flexWrap: { xs: "nowrap", sm: "wrap" },
              alignItems: { xs: "stretch", sm: "center" },
            }}
          >
            {canManage && <Button variant="contained" startIcon={<CloudUploadOutlined />} onClick={() =>
setUploadOpen(true)} sx={{ whiteSpace: "nowrap" }}>上传书籍</Button>}
            <FormControl size="small" sx={{ minWidth: 118 }}>
              <Select label="格式" value={format} onChange={(event: any) => updateParam("format", event.t
arget.value, "ALL")} startAdornment={<FilterListRounded sx={{ mr: 1, color: "text.secondary" }} />}>
                <MenuItem value="ALL">全部格式</MenuItem>
                <MenuItem value="EPUB">EPUB</MenuItem>
                <MenuItem value="PDF">PDF</MenuItem>
              </Select>
            </FormControl>
            <FormControl size="small" sx={{ minWidth: 126 }}>
              <Select label="排序" value={sort} onChange={(event: any) => updateParam("sort", event.targe
t.value, "recent")}>
                <MenuItem value="recent">最近入库</MenuItem>
                <MenuItem value="title">按书名</MenuItem>
                <MenuItem value="author">按作者</MenuItem>
              </Select>
            </FormControl>
          </Stack>
        </Stack>
        {booksQuery.isPending ? (
          <Box role="status" aria-label="正在加载藏书" sx={{ display: "grid", gridTemplateColumns: { xs:
"repeat(2, minmax(0, 1fr))", md: "repeat(4, minmax(0, 1fr))" }, gap: 3 }}>
            {Array.from({ length: 4 }, (_, index) => <Box key={index}><Box sx={{ aspectRatio: "2 / 3", bg
color: "action.hover", borderRadius: `${tokens.radius.lg}px`, mb: 2 }} /><Skeleton active title={false} p
aragraph={{ rows: 2 }} /></Box>)}
          </Box>
        ) : booksQuery.isError && !booksQuery.data ? (
          <Alert severity="error">书库暂时无法读取，请检查 API 与 NAS 状态。</Alert>
        ) : books.length === 0 ? (
          <Stack spacing={2} sx={{ alignItems: "center", textAlign: "center", py: 12 }}>
            <AutoAwesomeOutlined color="primary" sx={{ fontSize: 42 }} />
            <Typography variant="h4">没有找到匹配的书</Typography>
            <Typography color="text.secondary">试试作者、书名或标签，或者清除格式筛选。</Typography>
            <Button onClick={() => setParams({})}>清除筛选</Button>
          </Stack>
        ) : (
          <Box sx={{ display: "grid", gridTemplateColumns: { xs: "repeat(2, minmax(0, 1fr))", sm: "repeat
(3, minmax(0, 1fr))", md: "repeat(4, minmax(0, 1fr))", lg: "repeat(6, minmax(0, 1fr))" }, gap: { xs: 2, s
m: 3, md: 4 } }}>
            {bookEntries.map(({ item: book, pageIndex, itemIndex }) => (
              <PaginatedItemReveal key={book.id} animate={pageIndex > 0} order={itemIndex}>
```

```
                <BookCard
                  book={book}
                  onOpen={setSelectedBook}
                  onRead={(value) => navigate(`/reader/${value.id}`)}
                  onAddToBooklist={setBooklistBook}
                  onEditMetadata={canManage ? setMetadataBook : undefined}
                  onFileOperation={canManage ? (value, type) => setOperation({ book: value, type }) : und
efined}
                />
              </PaginatedItemReveal>
            ))}
          </Box>
        )}
        {(booksQuery.hasNextPage || booksQuery.isFetchingNextPage) && !booksQuery.isFetchNextPageError &&
 (
          <Stack ref={triggerRef} role="status" aria-live="polite" sx={{ minHeight: 96, alignItems: "cent
er", justifyContent: "center", mt: 3 }}>
            {booksQuery.isFetchingNextPage ? (
              <Stack direction="row" spacing={1.5} sx={{ alignItems: "center", color: "text.secondary" }}
>
                <Skeleton active title={false} paragraph={{ rows: 2 }} />
                <Typography variant="body2">正在展开更多藏书…</Typography>
              </Stack>
            ) : automaticLoadingSupported ? (
              <Typography variant="caption" color="text.disabled">继续向下浏览，将自动加载更多藏书</Typog
raphy>
            ) : (
              <Button variant="outlined" onClick={loadNextPage}>继续加载</Button>
            )}
          </Stack>
        )}
        {booksQuery.isFetchNextPageError && (
          <Alert
            severity="warning"
            sx={{ mt: 4 }}
            action={<Button color="inherit" onClick={() => void booksQuery.fetchNextPage()}>重新加载</But
ton>}
          >
            更多藏书加载失败，已经显示的书籍不受影响。
          </Alert>
        )}
        {!booksQuery.hasNextPage && books.length > 36 && (
          <Typography variant="caption" color="text.disabled" sx={{ display: "block", textAlign: "center"
, mt: 6 }}>已加载全部 {books.length} 本藏书</Typography>
        )}
      </PageContainer>
      <BookDetailsDialog
        book={selectedBook}
        canManage={canManage}
        onClose={() => setSelectedBook(null)}
        onRead={(book) => navigate(`/reader/${book.id}`)}
        onAddToBooklist={(book) => { setSelectedBook(null); setBooklistBook(book); }}
        onOperation={(book, type) => { setSelectedBook(null); setOperation({ book, type }); }}
        onEditMetadata={(book) => { setSelectedBook(null); setMetadataBook(book); }}
      />
      <AddToBooklistDialog
        open={Boolean(booklistBook)}
```

```
        bookId={booklistBook?.id ?? ""}
        bookTitle={booklistBook?.title ?? ""}
        onClose={() => setBooklistBook(null)}
        onCompleted={(message) => { setBooklistBook(null); setNotice(message); }}
      />
      <FileOperationDialog
        book={operation?.book ?? null}
        type={operation?.type ?? null}
        onClose={() => setOperation(null)}
        onCompleted={(result) => { setNotice(result.status === "SUCCEEDED" ? `任务已完成：${result.stage
?? result.status}` : "文件任务已提交，可在文件任务页查看进度"); void queryClient.invalidateQueries({ quer
yKey: ["books"] }); void queryClient.invalidateQueries({ queryKey: ["recycle-bin"] }); }}
      />
      <MetadataDialog
        book={metadataBook}
        onClose={() => setMetadataBook(null)}
        onSavedImmediately={setNotice}
        onSaveFailed={setNotice}
        onCompleted={(message) => { setNotice(message); void queryClient.invalidateQueries({ queryKey: ["
books"] }); }}
      />
      <BookUploadDialog open={uploadOpen} onClose={() => setUploadOpen(false)} onCompleted={(message) =>
{ setNotice(message); void queryClient.invalidateQueries({ queryKey: ["books"] }); }} />
      <Snackbar open={Boolean(notice)} autoHideDuration={4500} onClose={() => setNotice("")} message={not
ice} />
      {showBackToTop && (
        <Tooltip title="回到顶部">
          <Fab
            color="primary"
            size="medium"
            aria-label="回到顶部"
            onClick={() => window.scrollTo({ top: 0, behavior: "smooth" })}
            sx={{
              position: "fixed",
              right: { xs: `${tokens.spacing[4]}px`, sm: `${tokens.spacing[6]}px` },
              bottom: { xs: `${tokens.spacing[4]}px`, sm: `${tokens.spacing[6]}px` },
              zIndex: (theme: any) => theme.zIndex.fab,
              boxShadow: tokens.shadow.popover,
            }}
          >
            <KeyboardArrowUpRounded />
          </Fab>
        </Tooltip>
      )}
    </>
  );
}
function BookDetailsDialog({ book, canManage, onClose, onRead, onOperation, onEditMetadata, onAddToBookli
st }: {
  book: Book | null;
  canManage: boolean;
  onClose: () => void;
  onRead: (book: Book) => void;
  onAddToBooklist: (book: Book) => void;
  onOperation: (book: Book, type: FileOperationType) => void;
  onEditMetadata: (book: Book) => void;
```

```
}) {
  return (
    <Dialog
      open={Boolean(book)}
      onClose={onClose}
      fullWidth
      maxWidth="md"
      sx={{
        width: { xs: `calc(100% - ${tokens.spacing[4]}px)`, sm: `calc(100% - ${tokens.spacing[12]}px)` },
        maxWidth: { xs: "none", sm: `${tokens.layout.readingMax}px` },
        maxHeight: { xs: `calc(100% - ${tokens.spacing[4]}px)`, sm: `calc(100% - ${tokens.spacing[12]}px)
` },
        m: { xs: `${tokens.spacing[2]}px`, sm: `${tokens.spacing[6]}px` },
      }}
    >
      {book && (
        <>
          <DialogContent sx={{ p: { xs: `${tokens.spacing[3]}px`, sm: `${tokens.spacing[6]}px`, md: `${to
kens.spacing[8]}px` }, minHeight: 0 }}>
            <Box sx={{ display: "grid", gridTemplateColumns: { xs: "1fr", sm: "180px minmax(0, 1fr)", md:
 "210px minmax(0, 1fr)" }, gap: { xs: `${tokens.spacing[4]}px`, sm: `${tokens.spacing[6]}px`, md: `${toke
ns.spacing[8]}px` }, minWidth: 0 }}>
              <Box component="img" src={book.coverUrl} alt={`${book.title}封面`} sx={{ width: "100%", max
Width: { xs: `${tokens.spacing[24]}px`, sm: "none" }, mx: { xs: "auto", sm: 0 }, aspectRatio: "2 / 3", ob
jectFit: "cover", borderRadius: `${tokens.radius.lg}px`, boxShadow: tokens.shadow.cover }} />
              <Stack sx={{ alignItems: "flex-start", minWidth: 0 }}>
                <Stack direction="row" spacing={1} sx={{ mb: `${tokens.spacing[4]}px`, flexWrap: "wrap",
rowGap: `${tokens.spacing[2]}px` }}><Chip label={book.format} variant="outlined" size="small" />{book.tag
s.map((tag) => <Chip key={tag} label={tag} size="small" />)}</Stack>
                <Typography variant="h2" sx={{ fontSize: { xs: `${tokens.typography.fontSize.heading}px`,
 md: `${tokens.typography.fontSize.display}px` }, overflowWrap: "anywhere" }}>{book.title}</Typography>
                <Typography color="text.secondary" sx={{ mt: 1 }}>{book.author}</Typography>
                {book.description && <Typography sx={{ mt: `${tokens.spacing[6]}px` }}>{book.description}
</Typography>}
                <Divider flexItem sx={{ my: { xs: `${tokens.spacing[5]}px`, md: `${tokens.spacing[6]}px`
} }} />
                <Stack spacing={1.2} sx={{ width: "100%", minWidth: 0 }}>
                  <Meta icon={<ScheduleRounded />} label="入库时间" value={new Intl.DateTimeFormat("zh-CN
", { dateStyle: "long" }).format(new Date(book.addedAt))} />
                  <Meta icon={<MenuBookRounded />} label="文件位置" value={`${book.libraryRoot}/${book.re
lativePath}`} />
                </Stack>
              </Stack>
            </Box>
          </DialogContent>
          <DialogActions sx={{ px: { xs: `${tokens.spacing[3]}px`, sm: `${tokens.spacing[6]}px`, md: `${t
okens.spacing[8]}px` }, pt: { xs: `${tokens.spacing[2]}px`, sm: 0 }, pb: { xs: `${tokens.spacing[3]}px`,
sm: `${tokens.spacing[6]}px` }, display: { xs: "grid", sm: "flex" }, gridTemplateColumns: { xs: "repeat(2
, minmax(0, 1fr))" }, gap: `${tokens.spacing[2]}px`, alignItems: "stretch" }}>
            {canManage && (
              <Stack direction="row" spacing={1} sx={{ gridColumn: { xs: "1 / -1", sm: "auto" }, width: {
 xs: "100%", sm: "auto" }, flexWrap: "wrap" }}>
                <Button startIcon={<EditNoteRounded />} color="inherit" onClick={() => onEditMetadata(boo
k)} sx={{ flex: { xs: "1 1 auto", sm: "0 0 auto" } }}>编辑元信息</Button>
                <Button color="inherit" onClick={() => onOperation(book, "RENAME")} sx={{ flex: { xs: "1
1 auto", sm: "0 0 auto" } }}>管理文件</Button>
              </Stack>
            )}
            <Box sx={{ display: { xs: "none", sm: "block" }, flex: 1 }} />
            <Stack direction="row" spacing={1} sx={{ gridColumn: { xs: "1 / -1", sm: "auto" }, width: { x
s: "100%", sm: "auto" } }}>
              <Button onClick={onClose} color="inherit" sx={{ flex: { xs: 1, sm: "initial" } }}>关闭</But
ton>
              <Button startIcon={<BookmarkAddOutlined />} onClick={() => onAddToBooklist(book)} sx={{ fle
x: { xs: 1, sm: "initial" } }}>加入书单</Button>
              <Button variant="contained" onClick={() => onRead(book)} sx={{ flex: { xs: 1, sm: "initial"
 } }}>开始阅读</Button>
            </Stack>
          </DialogActions>
        </>
      )}
    </Dialog>
  );
```

```
}
function Meta({ icon, label, value }: { icon: ReactNode; label: string; value: string }) {
  return (
    <Stack direction="row" spacing={1.5} sx={{ alignItems: "flex-start", color: "text.secondary" }}>
      <Box sx={{ display: "inline-flex", mt: "2px" }}>{icon}</Box>
      <Box sx={{ minWidth: 0, overflowWrap: "anywhere" }}><Typography variant="caption">{label}</Typograp
hy><Typography variant="body2" color="text.primary" sx={{ overflowWrap: "anywhere" }}>{value}</Typography
></Box>
    </Stack>
  );
}
package com.elexvx.bookkin.ingestion.upload;
import static com.elexvx.bookkin.ingestion.upload.BookUploadModels.*;
import com.elexvx.bookkin.ai.AiMetadataService;
import com.elexvx.bookkin.config.BookKinProperties;
import tools.jackson.core.type.TypeReference;
import tools.jackson.databind.JsonNode;
import tools.jackson.databind.ObjectMapper;
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.time.Duration;
import java.time.OffsetDateTime;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.HexFormat;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import org.springframework.stereotype.Service;
@Service
public class MetadataEnrichmentService {
    private static final Set<String> COVER_HOSTS = Set.of("covers.openlibrary.org", "books.google.com", "
books.googleusercontent.com");
    private final BookUploadRepository uploads;
    private final BookKinProperties properties;
    private final AiMetadataService ai;
    private final ObjectMapper json;
    private final HttpClient http = HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(3)).followR
edirects(HttpClient.Redirect.NORMAL).build();
    public MetadataEnrichmentService(BookUploadRepository uploads, BookKinProperties properties, AiMetada
taService ai, ObjectMapper json) {
        this.uploads = uploads;
        this.properties = properties;
        this.ai = ai;
        this.json = json;
    }
    public EnrichmentResult enrich(MetadataDraft local) {
        return enrich(local, false, null, false);
```

```
    }
    public EnrichmentResult enrich(MetadataDraft local, boolean includeAi, String providerId, boolean for
ceAi) {
        List<MetadataCandidate> candidates = new ArrayList<>();
        if (properties.metadataProviders().openLibraryEnabled()) candidates.addAll(safely("OPEN_LIBRARY",
 () -> openLibrary(local)));
        if (properties.metadataProviders().googleApiKey() != null && !properties.metadataProviders().goog
leApiKey().isBlank()) {
            candidates.addAll(safely("GOOGLE_BOOKS", () -> google(local)));
        }
        MetadataDraft enriched = fillBlanks(local, candidates);
        if (includeAi && (forceAi || needsAiHelp(enriched, candidates))) {
            var result = ai.match(enriched, candidates, providerId);
            candidates.addAll(result.candidates());
            enriched = fillBlanks(enriched, result.candidates());
        }
        return new EnrichmentResult(enriched, List.copyOf(candidates));
    }
    public byte[] downloadCover(MetadataCandidate candidate, long maxBytes) throws Exception {
        if (candidate == null || candidate.provider() == MetadataSource.AI || candidate.coverUrl() == nul
l) {
            throw new IllegalArgumentException("候选项没有可下载的权威封面");
        }
        URI uri = URI.create(candidate.coverUrl().replace("http://", "https://"));
        assertCoverHost(uri);
        var request = HttpRequest.newBuilder(uri).timeout(Duration.ofSeconds(8)).header("User-Agent", "Bo
okKin/1.0").GET().build();
        var response = http.send(request, HttpResponse.BodyHandlers.ofByteArray());
        assertCoverHost(response.uri());
        if (response.statusCode() != 200 || response.body().length == 0 || response.body().length > maxBy
tes) {
            throw new IllegalArgumentException("在线封面不可用或超过大小限制");
        }
        String contentType = response.headers().firstValue("content-type").orElse("").toLowerCase(Locale.
ROOT);
        if (!contentType.startsWith("image/") || contentType.contains("svg")) throw new IllegalArgumentEx
ception("在线封面格式不安全");
        return response.body();
    }
    private List<MetadataCandidate> openLibrary(MetadataDraft draft) throws Exception {
        String query = draft.isbn() != null && !draft.isbn().isBlank()
                ? "isbn=" + encode(draft.isbn())
                : "title=" + encode(draft.title()) + "&author=" + encode(first(draft.authors()));
        String url = "https://openlibrary.org/search.json?" + query + "&limit=3&fields=key,title,subtitle
,author_name,publisher,first_publish_year,isbn,subject,cover_i";
        return cached("OPEN_LIBRARY", url, () -> {
            JsonNode root = getJson(url);
            List<MetadataCandidate> values = new ArrayList<>();
            for (JsonNode node : root.path("docs")) {
                String id = text(node, "key");
                String cover = node.hasNonNull("cover_i") ? "https://covers.openlibrary.org/b/id/" + node
.get("cover_i").asText() + "-L.jpg" : null;
                values.add(new MetadataCandidate(id, MetadataSource.OPEN_LIBRARY, text(node, "title"), te
xt(node, "subtitle"),
                        strings(node.path("author_name")), first(strings(node.path("publisher"))), intege
rText(node, "first_publish_year"),
                        first(strings(node.path("isbn"))), null, strings(node.path("subject")).stream().l
imit(20).toList(), cover));
            }
            return values;
        });
    }
    private List<MetadataCandidate> google(MetadataDraft draft) throws Exception {
```

```
        String q = draft.isbn() != null && !draft.isbn().isBlank()
                ? "isbn:" + draft.isbn()
                : "intitle:" + draft.title() + (draft.authors().isEmpty() ? "" : "+inauthor:" + draft.aut
hors().getFirst());
        String url = "https://www.googleapis.com/books/v1/volumes?q=" + encode(q) + "&maxResults=3&key="
+ encode(properties.metadataProviders().googleApiKey());
        return cached("GOOGLE_BOOKS", url.replace(properties.metadataProviders().googleApiKey(), "[KEY]")
, () -> {
            JsonNode root = getJson(url);
            List<MetadataCandidate> values = new ArrayList<>();
            for (JsonNode item : root.path("items")) {
                JsonNode node = item.path("volumeInfo");
                String isbn = null;
                for (JsonNode identifier : node.path("industryIdentifiers")) {
                    if (identifier.path("type").asText().startsWith("ISBN")) { isbn = identifier.path("id
entifier").asText(); break; }
                }
                String cover = text(node.path("imageLinks"), "thumbnail");
                values.add(new MetadataCandidate(text(item, "id"), MetadataSource.GOOGLE_BOOKS, text(node
, "title"), text(node, "subtitle"),
                        strings(node.path("authors")), text(node, "publisher"), text(node, "publishedDate
"), isbn,
                        text(node, "description"), strings(node.path("categories")), cover));
            }
            return values;
        });
    }
    private MetadataDraft fillBlanks(MetadataDraft local, List<MetadataCandidate> candidates) {
        String title = local.title();
        String subtitle = local.subtitle();
        List<String> authors = unknownAuthors(local.authors()) ? List.of() : local.authors();
        String publisher = local.publisher();
        String date = local.publishedDate();
        String isbn = local.isbn();
        String description = local.description();
        List<String> tags = local.tags();
        Map<String, MetadataSource> sources = new HashMap<>(local.sources());
        for (MetadataCandidate candidate : candidates) {
            if (blank(title) && !blank(candidate.title())) { title = candidate.title(); sources.put("titl
e", candidate.provider()); }
            if (blank(subtitle) && !blank(candidate.subtitle())) { subtitle = candidate.subtitle(); sourc
es.put("subtitle", candidate.provider()); }
            if (authors.isEmpty() && candidate.authors() != null && !candidate.authors().isEmpty()) { aut
hors = candidate.authors(); sources.put("authors", candidate.provider()); }
            if (blank(publisher) && !blank(candidate.publisher())) { publisher = candidate.publisher(); s
ources.put("publisher", candidate.provider()); }
            if (blank(date) && !blank(candidate.publishedDate())) { date = candidate.publishedDate(); sou
rces.put("publishedDate", candidate.provider()); }
            if (blank(isbn) && !blank(candidate.isbn())) { isbn = candidate.isbn(); sources.put("isbn", c
andidate.provider()); }
            if (blank(description) && !blank(candidate.description())) { description = candidate.descript
ion(); sources.put("description", candidate.provider()); }
            if (tags.isEmpty() && candidate.tags() != null && !candidate.tags().isEmpty()) { tags = candi
date.tags(); sources.put("tags", candidate.provider()); }
        }
        return new MetadataDraft(title, subtitle, authors.isEmpty() ? local.authors() : authors, local.tr
anslators(), local.language(), publisher, date, isbn,
                description, local.series(), local.seriesIndex(), tags, local.pageCount(), local.wordCoun
t(), sources, local.targetPath());
    }
    private boolean needsAiHelp(MetadataDraft local, List<MetadataCandidate> candidates) {
        if (candidates.isEmpty()) return true;
        String isbn = normalize(local.isbn());
        String title = normalize(local.title());
        String authors = normalize(String.join(" ", meaningfulAuthors(local.authors())));
        return candidates.stream().noneMatch(candidate -> {
```

```
            String candidateIsbn = normalize(candidate.isbn());
            if (!isbn.isBlank() && !candidateIsbn.isBlank()) return isbn.equals(candidateIsbn);
            String candidateTitle = normalize(candidate.title());
            String candidateAuthors = normalize(String.join(" ", candidate.authors() == null ? List.of()
: candidate.authors()));
            return !title.isBlank() && title.equals(candidateTitle) && (authors.isBlank() || candidateAut
hors.contains(authors) || authors.contains(candidateAuthors));
        });
    }
    private List<MetadataCandidate> cached(String provider, String cacheInput, CandidateSupplier supplier
) throws Exception {
        String hash = "sha256:" + HexFormat.of().formatHex(MessageDigest.getInstance("SHA-256").digest(ca
cheInput.getBytes(StandardCharsets.UTF_8)));
        String cached = uploads.cached(provider, hash);
        if (cached != null) return json.readValue(cached, new TypeReference<>() {});
        List<MetadataCandidate> value = supplier.get();
        uploads.cache(provider, hash, json.writeValueAsString(value), OffsetDateTime.now().plusDays(30));
        return value;
    }
    private List<MetadataCandidate> safely(String provider, CandidateSupplier supplier) {
        try { return supplier.get(); }
        catch (Exception ignored) { return List.of(); }
    }
    private JsonNode getJson(String url) throws Exception {
        var request = HttpRequest.newBuilder(URI.create(url)).timeout(Duration.ofSeconds(5)).header("User
-Agent", "BookKin/1.0").GET().build();
        var response = http.send(request, HttpResponse.BodyHandlers.ofString(StandardCharsets.UTF_8));
        if (response.statusCode() != 200) throw new IllegalStateException("metadata provider returned " +
 response.statusCode());
        return json.readTree(response.body());
    }
    private void assertCoverHost(URI uri) {
        String host = uri.getHost() == null ? "" : uri.getHost().toLowerCase(Locale.ROOT);
        if (!"https".equalsIgnoreCase(uri.getScheme()) || (!COVER_HOSTS.contains(host) && !host.endsWith(
".googleusercontent.com"))) {
            throw new IllegalArgumentException("在线封面来源不受信任");
        }
    }
    private String text(JsonNode node, String field) { return node.hasNonNull(field) && !node.get(field).
asText().isBlank() ? node.get(field).asText() : null; }
    private String integerText(JsonNode node, String field) { return node.has(field) && node.get(field).i
sNumber() ? node.get(field).asText() : null; }
    private List<String> strings(JsonNode node) {
        if (node == null || !node.isArray()) return List.of();
        List<String> values = new ArrayList<>();
        Set<String> seen = new HashSet<>();
        for (JsonNode item : node) if (item.isTextual() && seen.add(item.asText())) values.add(item.asTex
t());
        return values;
    }
    private String first(List<String> values) { return values == null || values.isEmpty() ? null : values
.getFirst(); }
    private boolean blank(String value) { return value == null || value.isBlank(); }
    private String normalize(String value) { return value == null ? "" : value.toLowerCase(Locale.ROOT).r
eplaceAll("[\\p{Punct}\\s]+", ""); }
    private List<String> meaningfulAuthors(List<String> values) {
        return values == null ? List.of() : values.stream().filter(value -> value != null && !value.isBla
nk() && !"未知作者".equals(value.strip())).toList();
    }
    private boolean unknownAuthors(List<String> values) { return meaningfulAuthors(values).isEmpty(); }
    private String encode(String value) { return URLEncoder.encode(value == null ? "" : value, StandardCh
arsets.UTF_8); }
    public record EnrichmentResult(MetadataDraft draft, List<MetadataCandidate> candidates) {}
    @FunctionalInterface private interface CandidateSupplier { List<MetadataCandidate> get() throws Excep
tion; }
```

```
}
package com.elexvx.bookkin.reading;
import com.elexvx.bookkin.audit.AuditService;
import com.elexvx.bookkin.common.ApiException;
import com.elexvx.bookkin.config.BookKinProperties;
import com.elexvx.bookkin.users.UserRepository;
import jakarta.annotation.PostConstruct;
import java.awt.Font;
import java.io.IOException;
import java.io.InputStream;
import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.file.Files;
import java.nio.file.LinkOption;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.nio.file.StandardOpenOption;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.Principal;
import java.util.HexFormat;
import java.util.List;
import java.util.Locale;
import java.util.UUID;
import org.springframework.core.io.FileSystemResource;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;
@Service
public class ReaderFontService {
    private final ReaderFontRepository fonts;
    private final UserRepository users;
    private final BookKinProperties properties;
    private final AuditService audit;
    private Path root;
    public ReaderFontService(ReaderFontRepository fonts, UserRepository users, BookKinProperties properti
es, AuditService audit) {
        this.fonts = fonts;
        this.users = users;
        this.properties = properties;
        this.audit = audit;
    }
    @PostConstruct
    void prepareStorage() {
        root = Path.of(properties.fonts().storagePath()).toAbsolutePath().normalize();
        try {
            ensureDirectory(root);
            ensureDirectory(root.resolve("staging"));
            ensureDirectory(root.resolve("content"));
        } catch (IOException exception) {
```

```
            throw new IllegalStateException("字体存储目录不可用", exception);
        }
    }
    public List<ReaderFontView> list(boolean includeDisabled) {
        return fonts.findAll(includeDisabled).stream().map(this::view).toList();
    }
    public ReaderFontView create(String displayName, String kind, String filename, long sizeBytes, String
 licenseNote, Principal principal) {
        UUID actor = actor(principal);
        Format format = format(filename);
        if (sizeBytes <= 0 || sizeBytes > properties.fonts().maxFileSize()) {
            throw new ApiException(HttpStatus.PAYLOAD_TOO_LARGE, "FONT_SIZE_LIMIT", "字体文件为空或超过大
小限制。");
        }
        String normalizedName = cleanName(displayName, filename);
        String normalizedKind = normalizeKind(kind);
        UUID id = UUID.randomUUID();
        String stagingPath = "staging/" + id + ".part";
        ReaderFontRepository.ReaderFont created = fonts.create(id, normalizedName, normalizedName, normal
izedKind,
                format.name(), format.mimeType(), sizeBytes, "pending:" + id, stagingPath, cleanLicense(l
icenseNote), actor);
        audit.record(actor, "READER_FONT_UPLOAD_STARTED", "READER_FONT", id.toString(), filename, staging
Path,
                null, null, "SUCCEEDED", "{\"format\":\"" + format.name() + "\"}");
        return view(created);
    }
    public ReaderFontView receive(UUID id, long contentLength, InputStream input, Principal principal) th
rows IOException {
        UUID actor = actor(principal);
        ReaderFontRepository.ReaderFont font = fonts.findById(id).orElseThrow(() -> ApiException.notFound
("FONT_NOT_FOUND", "未找到字体上传会话。"));
        if (font.contentPath() != null) return view(fonts.findById(id).orElseThrow());
        if (contentLength >= 0 && contentLength != font.sizeBytes()) {
            throw new ApiException(HttpStatus.BAD_REQUEST, "FONT_LENGTH_MISMATCH", "实际上传大小与字体会
话声明不一致。");
        }
        Path staging = safeChild(root, font.stagingPath());
        Path target = null;
        try {
            Files.deleteIfExists(staging);
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] buffer = new byte[1024 * 1024];
            byte[] signature = new byte[12];
            int signatureLength = 0;
            long written = 0;
            ensureDirectory(staging.getParent());
            try (FileChannel channel = FileChannel.open(staging, StandardOpenOption.CREATE_NEW, StandardO
penOption.WRITE)) {
                int read;
                while ((read = input.read(buffer)) >= 0) {
                    if (read == 0) continue;
                    written += read;
                    if (written > font.sizeBytes() || written > properties.fonts().maxFileSize()) throw n
ew IllegalArgumentException("字体上传超过声明大小");
                    if (signatureLength < signature.length) {
                        int copied = Math.min(signature.length - signatureLength, read);
                        System.arraycopy(buffer, 0, signature, signatureLength, copied);
                        signatureLength += copied;
                    }
```

```
                    digest.update(buffer, 0, read);
                    ByteBuffer bytes = ByteBuffer.wrap(buffer, 0, read);
                    while (bytes.hasRemaining()) channel.write(bytes);
                }
                channel.force(true);
            }
            if (written != font.sizeBytes()) throw new IllegalArgumentException("字体文件不完整");
            Format format = Format.valueOf(font.format());
            validateSignature(format, signature, signatureLength);
            String fingerprint = "sha256:" + HexFormat.of().formatHex(digest.digest());
            var duplicate = fonts.findByFingerprint(fingerprint);
            if (duplicate.isPresent() && !duplicate.get().id().equals(id) && duplicate.get().contentPath(
) != null) {
                Files.deleteIfExists(staging);
                fonts.delete(id);
                audit.record(actor, "READER_FONT_UPLOAD_DEDUPLICATED", "READER_FONT", duplicate.get().id(
).toString(), font.displayName(), duplicate.get().contentPath(), null, fingerprint, "SUCCEEDED", "{}");
                return view(duplicate.get());
            }
            target = safeChild(root, "content/" + fingerprint.substring("sha256:".length()) + "." + forma
t.extension());
            if (Files.exists(target, LinkOption.NOFOLLOW_LINKS)) {
                if (Files.isSymbolicLink(target) || !Files.isRegularFile(target, LinkOption.NOFOLLOW_LINK
S)) throw new IllegalArgumentException("字体目标文件不安全");
                if (!fingerprint.equals(fileFingerprint(target))) throw new IllegalArgumentException("字
体目标文件指纹冲突");
                Files.deleteIfExists(staging);
            } else {
                try {
                    Files.move(staging, target, StandardCopyOption.ATOMIC_MOVE);
                } catch (java.nio.file.AtomicMoveNotSupportedException unsupported) {
                    Files.move(staging, target);
                }
            }
            String familyName = internalFamilyName(target, format, font.familyName());
            fonts.received(id, written, fingerprint, root.relativize(target).toString().replace('\\', '/'
), familyName);
            audit.record(actor, "READER_FONT_UPLOADED", "READER_FONT", id.toString(), font.displayName(),
 root.relativize(target).toString(), null, fingerprint, "SUCCEEDED", "{\"sizeBytes\":" + written + "}");
            return view(fonts.findById(id).orElseThrow());
        } catch (ApiException exception) {
            deleteQuietly(staging);
            throw exception;
        } catch (Exception exception) {
            deleteQuietly(staging);
            audit.record(actor, "READER_FONT_UPLOADED", "READER_FONT", id.toString(), font.displayName(),
 font.stagingPath(), null, null, "FAILED", "{}");
            throw new ApiException(HttpStatus.BAD_REQUEST, "FONT_UPLOAD_FAILED", message(exception));
        }
    }
    public ReaderFontView setStatus(UUID id, String status, Principal principal) {
        UUID actor = actor(principal);
        String normalized = "ENABLED".equals(status) || "DISABLED".equals(status) ? status : "";
        if (normalized.isEmpty()) throw new ApiException(HttpStatus.BAD_REQUEST, "FONT_STATUS_INVALID", "
字体状态无效。");
        ReaderFontRepository.ReaderFont current = fonts.findById(id).orElseThrow(() -> ApiException.notFo
und("FONT_NOT_FOUND", "未找到字体。"));
        if ("ENABLED".equals(normalized) && current.contentPath() == null) throw new ApiException(HttpSta
tus.CONFLICT, "FONT_NOT_READY", "字体文件尚未完成上传。");
        ReaderFontRepository.ReaderFont updated = fonts.setStatus(id, normalized);
        audit.record(actor, "ENABLED".equals(normalized) ? "READER_FONT_ENABLED" : "READER_FONT_DISABLED"
, "READER_FONT", id.toString(), current.displayName(), current.contentPath(), current.fingerprint(), upda
ted.fingerprint(), "SUCCEEDED", "{}");
```

```
        return view(updated);
    }
    @Scheduled(fixedDelay = 60 * 60 * 1000L)
    public void cleanupStaleUploads() {
        if (root == null) return;
        fonts.findStagingBefore(java.time.OffsetDateTime.now().minusHours(24)).forEach(font -> {
            try {
                deleteQuietly(safeChild(root, font.stagingPath()));
                fonts.delete(font.id());
            } catch (RuntimeException ignored) {
                // A stale session is best-effort cleanup; a future run retries the exact path.
            }
        });
    }
    public FontContent content(UUID id) {
        ReaderFontRepository.ReaderFont font = fonts.findEnabledById(id).orElseThrow(() -> ApiException.n
otFound("FONT_NOT_FOUND", "字体不可用。"));
        Path path = safeChild(root, font.contentPath());
        try {
            if (Files.isSymbolicLink(path) || !Files.isRegularFile(path, LinkOption.NOFOLLOW_LINKS) || !f
ont.fingerprint().equals(fileFingerprint(path))) throw new IOException("字体文件校验失败");
            return new FontContent(new FileSystemResource(path), font.mimeType());
        } catch (IOException exception) {
            throw ApiException.notFound("FONT_NOT_FOUND", "字体文件不可用。");
        }
    }
    private ReaderFontView view(ReaderFontRepository.ReaderFont font) {
        return new ReaderFontView(font.id(), font.displayName(), font.familyName(), font.kind(), font.sou
rce(), font.status(), font.format(),
                font.contentPath() == null ? null : "/api/v1/reader-fonts/" + font.id() + "/content", fon
t.licenseNote(), font.createdAt());
    }
    private UUID actor(Principal principal) {
        return users.findByUsername(principal.getName()).orElseThrow().id();
    }
    private String normalizeKind(String value) {
        if (!"SERIF".equals(value) && !"SANS".equals(value)) throw new ApiException(HttpStatus.BAD_REQUES
T, "FONT_KIND_INVALID", "字体类型无效。");
        return value;
    }
    private String cleanName(String value, String filename) {
        String fallback = basename(filename).replaceFirst("\\.[^.]+$", "");
        String result = value == null || value.isBlank() ? fallback : value.strip();
        if (result.isEmpty() || result.length() > 160) throw new ApiException(HttpStatus.BAD_REQUEST, "FO
NT_NAME_INVALID", "字体展示名称无效。");
        return result;
    }
    private String cleanLicense(String value) {
        if (value == null || value.isBlank()) return null;
        String result = value.strip();
        if (result.length() > 2000) throw new ApiException(HttpStatus.BAD_REQUEST, "FONT_LICENSE_INVALID"
, "字体许可说明过长。");
        return result;
    }
    private String basename(String filename) {
        if (filename == null || filename.isBlank()) throw new ApiException(HttpStatus.BAD_REQUEST, "FONT_
FILENAME_INVALID", "字体文件名不能为空。");
        String normalized = filename.replace('\\', '/');
```

```
        String value = normalized.substring(normalized.lastIndexOf('/') + 1).strip();
        if (value.isEmpty() || value.contains("..")) throw new ApiException(HttpStatus.BAD_REQUEST, "FONT
_FILENAME_INVALID", "字体文件名无效。");
        return value;
    }
    private Format format(String filename) {
        String extension = basename(filename).substring(basename(filename).lastIndexOf('.') + 1).toUpperC
ase(Locale.ROOT);
        try {
            return Format.valueOf(extension);
        } catch (IllegalArgumentException exception) {
            throw new ApiException(HttpStatus.BAD_REQUEST, "FONT_FORMAT_UNSUPPORTED", "只支持 WOFF2、WOFF
、TTF 和 OTF 字体。");
        }
    }
    private void validateSignature(Format format, byte[] signature, int length) {
        String magic = new String(signature, 0, Math.min(4, length), java.nio.charset.StandardCharsets.US
_ASCII);
        boolean valid = switch (format) {
            case WOFF2 -> "wOF2".equals(magic);
            case WOFF -> "wOFF".equals(magic);
            case OTF -> "OTTO".equals(magic) || startsWith(signature, length, new byte[]{0, 1, 0, 0});
            case TTF -> startsWith(signature, length, new byte[]{0, 1, 0, 0}) || "true".equals(magic) ||
"typ1".equals(magic);
        };
        if (!valid) throw new ApiException(HttpStatus.BAD_REQUEST, "FONT_SIGNATURE_INVALID", "字体文件签
名与扩展名不匹配。");
    }
    private boolean startsWith(byte[] input, int length, byte[] expected) {
        if (length < expected.length) return false;
        for (int index = 0; index < expected.length; index += 1) if (input[index] != expected[index]) ret
urn false;
        return true;
    }
    private String internalFamilyName(Path path, Format format, String fallback) {
        if (format == Format.WOFF || format == Format.WOFF2) return fallback;
        try {
            Font font = Font.createFont(Font.TRUETYPE_FONT, path.toFile());
            String name = font.getFamily(Locale.SIMPLIFIED_CHINESE);
            return name == null || name.isBlank() ? fallback : name;
        } catch (Exception ignored) {
            return fallback;
        }
    }
    private void ensureDirectory(Path path) throws IOException {
        if (Files.exists(path, LinkOption.NOFOLLOW_LINKS)) {
            if (Files.isSymbolicLink(path) || !Files.isDirectory(path, LinkOption.NOFOLLOW_LINKS)) throw
new IOException("字体目录不安全");
        } else Files.createDirectories(path);
        if (!path.toRealPath().startsWith(root == null ? path.toAbsolutePath().normalize() : root)) throw
 new IOException("字体目录越界");
    }
    private Path safeChild(Path base, String relative) {
        if (relative == null || relative.isBlank()) throw new ApiException(HttpStatus.NOT_FOUND, "FONT_NO
T_FOUND", "字体文件不存在。");
        Path candidate = base.resolve(relative).normalize();
        if (!candidate.startsWith(base.normalize())) throw new ApiException(HttpStatus.BAD_REQUEST, "FONT
_PATH_INVALID", "字体路径无效。");
        return candidate;
    }
    private String fileFingerprint(Path path) throws IOException {
```

```
        try (InputStream input = Files.newInputStream(path)) {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] buffer = new byte[1024 * 1024];
            int read;
            while ((read = input.read(buffer)) >= 0) if (read > 0) digest.update(buffer, 0, read);
            return "sha256:" + HexFormat.of().formatHex(digest.digest());
        } catch (NoSuchAlgorithmException impossible) {
            throw new IllegalStateException(impossible);
        }
    }
    private void deleteQuietly(Path path) {
        if (path == null) return;
        try { Files.deleteIfExists(path); } catch (IOException ignored) { }
    }
    private String message(Exception exception) { return exception.getMessage() == null ? "字体上传失败"
: exception.getMessage(); }
    public record ReaderFontView(UUID id, String displayName, String familyName, String kind, String sour
ce,
                                 String status, String format, String contentUrl, String licenseNote,
                                 java.time.OffsetDateTime createdAt) {}
    public record FontContent(FileSystemResource resource, String mimeType) {}
    private enum Format {
        WOFF2("woff2", "font/woff2"), WOFF("woff", "font/woff"), TTF("ttf", "font/ttf"), OTF("otf", "font
/otf");
        private final String extension;
        private final String mimeType;
        Format(String extension, String mimeType) { this.extension = extension; this.mimeType = mimeType;
 }
        String extension() { return extension; }
        String mimeType() { return mimeType; }
    }
}
import type { AnnotationColor, HighlightColor } from "./types";
import { tokens } from "../theme/generated-tokens";
export const highlightColorPresets: ReadonlyArray<{ value: HighlightColor; label: string; hex: string }>
= [
  { value: "YELLOW", label: "黄色", hex: tokens.color.primitive.highlightYellow },
  { value: "GREEN", label: "绿色", hex: tokens.color.primitive.highlightGreen },
  { value: "PINK", label: "粉色", hex: tokens.color.primitive.highlightPink },
  { value: "BLUE", label: "蓝色", hex: tokens.color.primitive.highlightBlue },
  { value: "ORANGE", label: "橙色", hex: tokens.color.primitive.highlightOrange },
];
export const READABLE_HIGHLIGHT_OPACITY = 0.34;
const legacyColorMap: Record<Exclude<AnnotationColor, HighlightColor>, HighlightColor> = {
  CORAL: "ORANGE",
  GOLD: "YELLOW",
  TEAL: "GREEN",
};
export function annotationColorHex(color?: AnnotationColor | string) {
  const normalized = color && color in legacyColorMap
    ? legacyColorMap[color as keyof typeof legacyColorMap]
    : color;
  return highlightColorPresets.find((preset) => preset.value === normalized)?.hex ?? highlightColorPreset
s[0].hex;
}
export function annotationColorBackground(color?: AnnotationColor | string, opacity = READABLE_HIGHLIGHT_
OPACITY) {
```

```
  const alpha = Math.round(Math.max(0, Math.min(1, opacity)) * 255).toString(16).padStart(2, "0");
  return `${annotationColorHex(color)}${alpha}`;
}
package com.elexvx.bookkin.catalog;
import java.util.List;
public record BrowseBookPage(List<BrowseBook> items, String nextCursor) {
}
import { AddRounded } from "@/ui/icons";
import { LibraryAddOutlined } from "@/ui/icons";
import { Alert } from "@/ui/feedback";
import { Box } from "@/ui/primitives";
import { Button } from "@/ui/buttons";
import { CircularProgress } from "@/ui/feedback";
import { Dialog } from "@/ui/overlays";
import { DialogActions } from "@/ui/overlays";
import { DialogContent } from "@/ui/overlays";
import { DialogTitle } from "@/ui/overlays";
import { List } from "@/ui/primitives";
import { ListItem } from "@/ui/primitives";
import { ListItemText } from "@/ui/primitives";
import { Stack } from "@/ui/primitives";
import { Typography } from "@/ui/primitives";
import { useQuery, useQueryClient } from "@tanstack/react-query";
import { useState } from "react";
import { api } from "../api/client";
import type { BooklistDetail } from "../domain/types";
import { tokens } from "../theme/generated-tokens";
import { VisibilityChip } from "./CatalogDiscoveryCards";
import { BooklistFormDialog } from "./BooklistFormDialog";
export function AddToBooklistDialog({ open, bookId, bookTitle, onClose, onCompleted }: {
  open: boolean;
  bookId: string;
  bookTitle: string;
  onClose: () => void;
  onCompleted: (message: string) => void;
}) {
  const queryClient = useQueryClient();
  const [busyId, setBusyId] = useState<string | null>(null);
  const [error, setError] = useState("");
  const [createOpen, setCreateOpen] = useState(false);
  const listsQuery = useQuery({
    queryKey: ["booklists", "mine", bookId],
    queryFn: () => api.listBooklists(),
    enabled: open,
  });
  const personalLists = listsQuery.data?.items.filter((list) => list.kind === "PERSONAL" && list.ownedByV
iewer) ?? [];
  const add = async (booklist: BooklistDetail) => {
    setBusyId(booklist.id);
    setError("");
    try {
```

```
      await api.addBooklistBooks(booklist.id, [bookId], booklist.revision);
      await queryClient.invalidateQueries({ queryKey: ["booklists"] });
      onCompleted(`已将《${bookTitle}》加入“${booklist.title}”`);
      onClose();
    } catch (reason) {
      setError(reason instanceof Error ? reason.message : "加入书单失败");
    } finally {
      setBusyId(null);
    }
  };
  const created = async (booklist: BooklistDetail, _message: string) => {
    setCreateOpen(false);
    await add(booklist);
  };
  return (
    <>
      <Dialog open={open && !createOpen} onClose={busyId ? undefined : onClose} fullWidth maxWidth="sm">
        <DialogTitle>将《{bookTitle}》加入书单</DialogTitle>
        <DialogContent>
          <Stack sx={{ gap: `${tokens.spacing[4]}px` }}>
            {error ? <Alert severity="error">{error}</Alert> : null}
            {listsQuery.isPending ? (
              <Stack sx={{ alignItems: "center", py: `${tokens.spacing[8]}px` }}><CircularProgress /></St
ack>
            ) : listsQuery.isError ? (
              <Alert severity="error">个人书单暂时无法读取，请重试。</Alert>
            ) : personalLists.length ? (
              <List sx={{ p: 0, border: 1, borderColor: "divider", borderRadius: `${tokens.radius.lg}px`,
 overflow: "hidden" }}>
                {personalLists.map((booklist, index) => (
                  <ListItem key={booklist.id} divider={index < personalLists.length - 1} secondaryAction=
{
                    <Button size="small" startIcon={<LibraryAddOutlined />} disabled={Boolean(busyId)} on
Click={() => void add(booklist)}>
                      加入
                    </Button>
                  }>
                    <ListItemText primary={booklist.title} secondary={<Stack component="span" direction="
row" sx={{ alignItems: "center", gap: `${tokens.spacing[2]}px`, mt: `${tokens.spacing[1]}px` }}><Visibili
tyChip visibility={booklist.visibility} /><Typography component="span" variant="caption" color="text.seco
ndary">{booklist.bookCount} 本</Typography></Stack>} />
                  </ListItem>
                ))}
              </List>
            ) : (
              <Box sx={{ textAlign: "center", py: `${tokens.spacing[8]}px` }}>
                <Typography variant="h5">还没有个人书单</Typography>
                <Typography color="text.secondary" sx={{ mt: `${tokens.spacing[2]}px` }}>先创建一个，再把
这本书收进去。</Typography>
              </Box>
            )}
            <Button variant={personalLists.length ? "outlined" : "contained"} startIcon={<AddRounded />}
onClick={() => setCreateOpen(true)}>新建个人书单</Button>
          </Stack>
        </DialogContent>
        <DialogActions><Button onClick={onClose} disabled={Boolean(busyId)}>取消</Button></DialogActions>
      </Dialog>
      <BooklistFormDialog
        open={open && createOpen}
```

```
        canCreateOfficial={false}
        onClose={() => setCreateOpen(false)}
        onSaved={(booklist, message) => void created(booklist, message)}
      />
    </>
  );
}
package com.elexvx.bookkin.common;
import org.springframework.http.HttpStatus;
public final class ApiException extends RuntimeException {
    private final HttpStatus status;
    private final String code;
    public ApiException(HttpStatus status, String code, String message) {
        super(message);
        this.status = status;
        this.code = code;
    }
    public HttpStatus status() { return status; }
    public String code() { return code; }
    public static ApiException conflict(String code, String message) {
        return new ApiException(HttpStatus.CONFLICT, code, message);
    }
    public static ApiException badRequest(String code, String message) {
        return new ApiException(HttpStatus.BAD_REQUEST, code, message);
    }
    public static ApiException notFound(String code, String message) {
        return new ApiException(HttpStatus.NOT_FOUND, code, message);
    }
    public static ApiException forbidden(String code, String message) {
        return new ApiException(HttpStatus.FORBIDDEN, code, message);
    }
}
package com.elexvx.bookkin.filemanagement;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import java.time.OffsetDateTime;
import java.util.List;
import java.util.Map;
import java.util.UUID;
public final class FileOperationModels {
    private FileOperationModels() {}
    public record PreviewRequest(
            UUID bookFileId,
            @NotNull FileOperationType type,
            UUID targetRootId,
            @Size(max = 2048) String targetPath,
            @NotNull String expectedFingerprint,
            Map<String, Object> metadata,
            UUID recycleBinEntryId) {}
    public record ExecuteRequest(@NotNull UUID previewToken, @NotNull String expectedFingerprint) {}
```

```
    public record FileConflict(String code, String message, String path) {}
    public record Preview(
            UUID previewToken,
            FileOperationType type,
            String sourcePath,
            String targetPath,
            long requiredBytes,
            String expectedFingerprint,
            OffsetDateTime expiresAt,
            List<FileConflict> conflicts,
            List<String> warnings) {}
    public record Operation(
            UUID id,
            FileOperationType type,
            FileOperationStatus status,
            String sourcePath,
            String targetPath,
            OffsetDateTime createdAt,
            String stage,
            String errorCode,
            String errorDetail) {}
}
package com.elexvx.bookkin.ai;
import com.elexvx.bookkin.config.BookKinProperties.AiProviderType;
import java.time.OffsetDateTime;
import java.util.List;
import java.util.Optional;
import java.util.UUID;
import org.jooq.DSLContext;
import org.jooq.Record;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;
@Repository
public class AiSettingsRepository {
    private final DSLContext dsl;
    public AiSettingsRepository(DSLContext dsl) {
        this.dsl = dsl;
    }
    public Optional<GlobalSettings> global() {
        return dsl.fetchOptional("select * from ai_settings where id = 1").map(this::mapGlobal);
    }
    public List<ProviderSettings> providers() {
        return dsl.fetch("select * from ai_provider_settings order by provider_id").map(this::mapProvider
);
    }
    @Transactional
    public void save(GlobalSettings global, List<ProviderSettings> providers, UUID actor) {
        dsl.execute("""
                insert into ai_settings(id, enabled, auto_match, max_candidates, timeout_seconds, updated
_by, updated_at)
                values (1, ?, ?, ?, ?, ?, now())
                on conflict (id) do update set enabled = excluded.enabled, auto_match = excluded.auto_mat
ch,
```

```
                  max_candidates = excluded.max_candidates, timeout_seconds = excluded.timeout_seconds,
                  updated_by = excluded.updated_by, updated_at = now()
                """, global.enabled(), global.autoMatch(), global.maxCandidates(), global.timeoutSeconds(
), actor);
        for (ProviderSettings provider : providers) {
            dsl.execute("""
                    insert into ai_provider_settings(provider_id, label, provider_type, enabled, base_url
,
                      api_key_ciphertext, model, updated_by, updated_at)
                    values (?, ?, ?, ?, ?, ?, ?, ?, now())
                    on conflict (provider_id) do update set label = excluded.label, provider_type = exclu
ded.provider_type,
                      enabled = excluded.enabled, base_url = excluded.base_url, api_key_ciphertext = excl
uded.api_key_ciphertext,
                      model = excluded.model, updated_by = excluded.updated_by, updated_at = now()
                    """, provider.id(), provider.label(), provider.type().name(), provider.enabled(), pro
vider.baseUrl(),
                    provider.apiKeyCiphertext(), provider.model(), actor);
        }
    }
    private GlobalSettings mapGlobal(Record row) {
        return new GlobalSettings(
                Boolean.TRUE.equals(row.get("enabled", Boolean.class)),
                Boolean.TRUE.equals(row.get("auto_match", Boolean.class)),
                row.get("max_candidates", Integer.class),
                row.get("timeout_seconds", Integer.class),
                row.get("updated_at", OffsetDateTime.class));
    }
    private ProviderSettings mapProvider(Record row) {
        return new ProviderSettings(
                row.get("provider_id", String.class),
                row.get("label", String.class),
                AiProviderType.valueOf(row.get("provider_type", String.class)),
                Boolean.TRUE.equals(row.get("enabled", Boolean.class)),
                row.get("base_url", String.class),
                row.get("api_key_ciphertext", String.class),
                row.get("model", String.class));
    }
    public record GlobalSettings(boolean enabled, boolean autoMatch, int maxCandidates, int timeoutSecond
s,
                                 OffsetDateTime updatedAt) {}
    public record ProviderSettings(String id, String label, AiProviderType type, boolean enabled, String
baseUrl,
                                   String apiKeyCiphertext, String model) {}
}
```